Koala: Cross-Substrate Tactical Representations and Zero-Shot Transfer in Graph-Based Go

Working draft (v3, narrowed after an internal referee pass + a decisive feature-leakage control; §5.2 now reports the matched-compute, multi-seed behavioral study). Numbers from `scripts/paper_tables.py --all`*,* `scripts/probe_decider.py`*, and* `scripts/multiseed_behavioral.py` *(*`results/paper/`*,* `results/behavioral/`*, deterministic, checkpoint SHAs in* `manifest.json`*). Target: TMLR / a games-or-GNN workshop.*

Abstract

Convolutional Go agents assume square-grid adjacency and do not directly extend to boards with different neighbourhood structure. We study Go on graph-structured boards using a **coordinate-free, topology-aware** graph neural network trained by self-play on a mixture of five substrate families, and ask two questions: (i) are local tactical variables *represented consistently* across graphs, and (ii) does *play* transfer to held-out board families, including a finite diamond-cubic lattice?

Regarding (i), rigorous controls reveal that cross-substrate decodability is **largely architectural rather than learned**. Tactical concepts — atari, capture, self-atari, and a locally-surrounded empty point (an "eye-like" pattern, not necessarily a true eye) — are highly decodable across substrates, but comparing raw input features, an untrained network, and a trained network shows that standard message-passing accounts for most of the representational gain: atari is 100% recoverable from the input liberty features alone, and for the remaining concepts an *untrained* network's gain over the raw inputs is nearly the same as a trained one's (e.g. capture +0.18 vs +0.19 AUC), leaving only a small, concept-dependent learned increment (largest for self-atari, ≈4.6× the across-init std). A square-only specialist matches the mixture on cross-substrate alignment. We therefore claim cross-substrate *decodability* of tactical information, not the emergence of learned substrate-invariant concepts.

For (ii) we train mixture, square-specialist, and diamond-cubic-specialist agents from scratch under matched self-play game and search-simulation budgets, using three training seeds. In all three observed seed pairings, the mixture defeats the square specialist on a held-out Penrose r8 patch and on the unseen diamond-cubic graph family. The Penrose result tests same-family patch-scale generalization (Penrose r5 was present during training); **diamond-cubic is the clean unseen-family transfer result**. Performance on the unseen planar trihexagonal family and on the square family is inconclusive at this sample size. On diamond-cubic the mixture's mean win rate against a non-learned MCTS floor is 82%, with two decisive runs and one approximately even run. Because three seeds are insufficient for precise estimation of training-run variance, we treat the consistency across the observed runs as preliminary rather than statistically conclusive. We verify the evaluation diamond graph is genuinely non-planar (a $K_{3,3}$-subdivision certificate), report where transfer breaks under continued training, and release a one-command, deterministic reproduction. The agent is weak in absolute terms (∼8–10 kyu); the contribution is the cross-substrate decodability/transfer *characterization*, with controlled baselines, not strength.

---

1. Introduction

Go's rules reference only points and their adjacencies — never coordinates. Yet strong engines (AlphaGo, AlphaZero, KataGo) encode the square grid into a convolutional stack: their *learned filters* assume square-grid neighbourhoods and do not transfer to hexagonal, aperiodic, or arbitrary graph boards. We instead make the board **data**: a graph, with stones on nodes, played by a coordinate-free graph network with the same weights on any graph.

This is a clean setting for a structural-generalization question: when an agent is trained across a mixture of board topologies, (i) is tactical information *represented consistently* across them, and (ii) does behaviour *transfer* to unseen topologies? We answer both with explicit controls, and are careful to separate what the architecture and engineered inputs already provide from what training adds. Our headline out-of-distribution test is a non-planar diamond-cubic lattice, chosen deliberately: its interior is **4-regular**, so each interior stone has the same liberty budget as on a square grid and Go's capture economics carry over — making it a genuine topology change rather than a change of local degree.

Contributions: 1. A coordinate-free GNN+MCTS Go agent that plays arbitrary Euclidean tilings **and a verified non-planar diamond-cubic 3-D lattice** with one weight set, numerically consistent across three runtimes. 2. A **feature-leakage-controlled** probe analysis showing cross-substrate decodability of tactical variables is largely architectural, with only a small learned increment and no mixture-vs-specialist representational advantage — a deliberately narrowed representational claim. 3. A **behavioral transfer** result with real baselines, multi-seed and matched-budget (a non-learned MCTS floor; from-scratch square specialist and diamond-cubic oracle over three seeds): across all three observed seeds the mixture beats the square specialist on an **unseen non-planar diamond-cubic** family and on a **same-family** larger-Penrose patch, with inconclusive *square-tax* (the performance penalty on square boards from training on a mixture rather than specializing) and planar-trihexagonal comparisons, plus an account of where transfer breaks.

2. Related work

We position this work against four streams, and are explicit about which ideas are *not* ours.

GNNs for board games. Treating a board as a graph and learning over it with a GNN is established. Ben-Assayag and El-Yaniv (*Train on Small, Play the Large*, arXiv:2107.08387, 2021) pair a GNN with AlphaZero, view the board as a graph, and learn multiple games without domain knowledge — but their result is *size* scaling on familiar board families, not transfer across local topologies. Keller et al. (arXiv:2311.13414, 2023) argue GNNs match Hex's relational structure better than grid CNNs, in a single-game/single-topology study. Rigaux and Kashima (arXiv:2410.23753, 2024) replace AlphaZero's board tensor with a graph representation for chess and fine-tune 5×5→8×8; Gunawan, Ruan, and Huang (AAMAS 2022) give a game-agnostic GNN reasoner over Game Description Language. The GNN+PUCT core here is therefore **not novel**; what differs is the *scope of substrate heterogeneity* (§ below).

Transfer across sizes and shapes. Variable-board-size generalization is well established: KataGo (Wu, arXiv:1902.10565, 2019) randomizes board sizes during training; Gao, Yan, Hayward, and Müller (*A Transferable Neural Network for Hex*, ICGA Journal, 2018) show size-independent Hex networks transfer zero-shot across sizes without fine-tuning; AlphaViT (Fujita, PeerJ CS / arXiv:2408.13871, 2025) handles multiple games and variable sizes with one shared-weight network. These are the closest threats to any transfer claim — but all stay within a single rule-set on a single adjacency *family* (rectangular grids, hex boards). Transfer across genuinely *different adjacency*

structures, especially from planar tilings to a non-planar 3-D lattice, is the open regime we target. Gao et al. in particular bounds our contribution 3: zero-shot *same-family size* transfer is already done.

Go on non-standard boards. The *game* generalizes long before this work. Viennot's *Go on Graphs* specifies liberties, groups, captures, ko and passing directly in graph terms; Browne (*Go without Ko on Hexagonal Grids*, 2012) studies how Go mechanics change under altered adjacency; the `arbitraryGo` engine plays Go on arbitrary graphs; and 3-D Go on a cubic lattice exists in both software (`lene/go-3`) and Go-variant circles (Bob Hearn's 3-D Go, reported by the AGA, 2004). We therefore **do not** claim non-standard-board Go as a new idea. We use **diamond-cubic** (4-regular interior) rather than the degenerate degree-6 cubic lattice so capture economics resemble planar Go; the contribution is a *learned, transferring agent* over this space, not the ruleset. (Penrose-tiling Go appears only as community proposals we could find no maintained engine for; our Penrose instance may be new in implementation, but we make no priority claim.)

Probing game-playing networks. McGrath et al. (*Acquisition of Chess Knowledge in AlphaZero*, PNAS / arXiv:2111.09259, 2022) is the canonical concept-probe study for self-play agents. Crucially, the *methodological lesson behind our § 5.1 — that decodability is not evidence of a learned representation — is already established*: Hewitt and Liang (*Control Tasks*, EMNLP 2019) show probe accuracy alone is ambiguous; Voita and Titov (*MDL probing*, EMNLP / arXiv:2003.12298, 2020) report that standard probes "do not substantially favour pretrained over randomly initialized representations" — almost exactly our untrained-vs-trained GNN finding; and in the game domain, Pálsson and Björnsson (ECAI 2024) and Lovering et al. (*Concepts in AlphaZero in Hex*, NeurIPS 2022) both warn that probing performance alone is insufficient. Our § 5.1 is thus an **instantiation** of a known critique in cross-substrate self-play Go (with a raw-input vs untrained-GNN vs trained-GNN control stack), not a new methodological discovery — and we frame it as such.

Where that leaves us. The defensible novelty is the *regime*, not the ingredients: a single self-play-trained, coordinate-free agent spanning heterogeneous Euclidean tilings, an aperiodic tiling, and a non-planar 3-D lattice, plus an honest cross-substrate probe study showing the representational story is mostly architectural. We pitch the paper as an empirical study of topology-general Go and the limits of probe-based abstraction claims — not a new learning paradigm.

3. Method

Board as graph. A tiling compiler emits a `BoardGraph` (deterministic node order) from a patch's 1-skeleton; the rules engine is Tromp–Taylor (area scoring, positional superko) over arbitrary adjacency, cross-checked against a classical reference.

Network. A graph network (0.78M params, hidden 96, 8 message-passing blocks) with 42 per-node features — game features (own/opponent stone, chain liberties, legality, recent moves), structure features (degree one-hot, boundary distance, Laplacian positional encodings), and global scalars. Each block updates a node from the **mean and max** of its neighbours plus a whole-board summary; these aggregators are permutation- and degree-flexible, which is what lets one weight set run on any graph. Heads: policy, value, ownership. We call this *coordinate-free and topology-aware* — it consumes adjacency, degree, and positional encodings, so it is not "blind" to structure.

Positional encodings at inference (zero-shot). The 16 positional features are the first 16 non-trivial eigenvectors of the *unseen graph's own* symmetric-normalized Laplacian ($L = I - D^{\{-1/2\}AD}\{-1/2\}$), recomputed from scratch per board via a dense eigendecomposition at load time — trivial for our sizes ($n \leq 221$; sub-millisecond) and requiring **no spectral alignment** to any training graph. The eigenvector sign ambiguity is handled by **sign-flip augmentation during training** (signs are

randomized each sample), so the network is trained to be sign-invariant and the arbitrary signs on a new graph are benign; graphs with fewer than 17 nodes are zero-padded.

Search/training. PUCT MCTS guided by the network (AlphaZero-style); pure self-play from random weights across a mixture of substrates in one replay buffer. The same weights run in PyTorch, a C++ engine, and WebAssembly, **numerically agreeing to $\sim 10^{-6}$**.

4. Experimental setup

Frozen split (`tilinggo/experiments/splits.py`; selection touches the training pool only): - *Training:* square 8×8 (N=64), hexagonal (54), triangular (73), snub-square (56), Penrose r5 (86). - *Held-out:* square 7×7 (49), trihexagonal (54), Penrose r8 (221), **diamond-cubic 3-D (51)**.

Diamond graph (verified). The evaluation patch has **51 nodes, 73 edges**, degree distribution {deg 2: 24, deg 3: 10, deg 4: 17}. `networkx` certifies it is **non-planar** by exhibiting a Kuratowski counterexample — a subdivision of $K_{3,3}$ (6 branch vertices of degree 3; 19 nodes, 22 edges after subdivision); we release the edge list and this certificate (girth, reported previously, is descriptive and does not bear on planarity). Caveat: it is boundary-heavy — only 17/51 nodes are degree-4 interior (24 are degree-2 boundary), so "same liberty budget as square Go" holds for the *interior* only. **All principal results in this paper — probes (§5.1), the behavioral duels (§5.2), the floor and oracle matches, and the released-champion sweep — use this single 51-node patch** (`diamond_c2`, `diamond.generate(cells=2)`). A larger cells=3 patch (197 nodes, 109 interior) is noted only as a recommended instance for future interior-behaviour and degree-matched-rewiring controls; it is **not** used for any number reported here. `[TODO: cells=3 instance + a degree-matched rewired-graph control to separate local-degree effects from substrate.]`

Baselines. A non-learned **score-heuristic MCTS floor** (uniform priors + area-margin value), run as a `"HEURISTIC"` opponent inside the C++ duel binary (parity-verified vs the Python evaluator; heuristic-vs-heuristic ≈ 50%); and per-substrate **specialist** networks. Duels are opening-randomized (6 tempered plies), colors alternate, with 95% Wilson intervals.

Probes. Per substrate we sample states from random legal playouts and label four rule-derived concepts per node: atari, capture, self-atari, and a **locally-surrounded empty point** (empty node with all neighbours one colour — we deliberately do *not* call this an "eye," since on a general graph such a point may be a false eye or merely surrounded; it is an eye-*like* local pattern). Linear probes are fit with **state-grouped** train/test splits (GroupKFold — no within-position leakage), standardized on train; reported AUC is the mean of per-fold out-of-fold AUC, pooled across the three probe substrates, for a single dataset seed (no across-model or across-seed uncertainty — see §5.1). For the leakage control we probe three feature sets: the raw 42-dim input, the 15 game features only, and the final-layer hidden activation.

5. Results

5.1 The representational claim, controlled

Table 1 — same-substrate probe AUC by feature set (and the learned residual = hidden − input).

concept	input (42-d)	game-only	hidden (mixture)	hidden (random, K=5)	Δ(mix − rand)
atari	**1.000 ± .000**	1.000 ± .000	1.000 ± .000	0.998 ± .001	+0.002
capture	0.810 ± .053	0.500 ± .000	0.999 ± .001	0.987 ± .007	+0.012
self-atari	0.781 ± .026	0.500 ± .000	0.991 ± .004	0.945 ± .010	**+0.046**
surrounded-point (eye-like)	0.897 ± .047	0.700 ± .023	1.000 ± .001	0.982 ± .006	+0.018

AUCs are mean ± standard deviation (one dataset seed): for input/game/hidden(mixture) the spread is over the GroupKFold-by-state folds (probe stability); for hidden(random) it is over **five random initializations**, which is the relevant uncertainty for judging whether a learned increment exceeds init noise. Two conclusions, both cautionary: 1. **Atari is fully contained in the inputs** (a function of the liberty features); its high cross-substrate decodability is not learned (Δ +0.002, within init noise). 2. For the other concepts the hidden layer beats the raw input, but **an untrained network's hidden layer is already almost as decodable as a trained one's** (random-init hidden reaches 0.95–0.99 AUC). Most of the improvement over raw inputs is therefore **architectural** (permutation-invariant neighbour aggregation). The residual learned increment is small and *concept-dependent*: for self-atari it is +0.046, ≈4.6× the across-init standard deviation (a small but reproducible learned effect); for the surrounded-point concept +0.018 (≈3×); for capture +0.012 it is marginal (≈1.7×). So training does add a modest, concept-specific increment on top of a large architectural floor — not the absence of learning, but far from a learned substrate-invariant abstraction.

Consistent with this, axis-alignment A(f,S) is already high for an untrained network (0.786 ± 0.049, 5 seeds) and rises only modestly with training (mixture 0.953 ± 0.011, square specialist 0.933 ± 0.045 — overlapping; the specialist's square→Penrose probe transfer 0.968 is *higher* than the mixture's 0.965). **There is no mixture-vs-specialist representational advantage at this scale.** We therefore report cross-substrate *decodability* of tactical variables, largely architectural, and explicitly do **not** claim learned substrate-invariant concept emergence. The underlying caution — that probe decodability need not reflect a learned representation, and that randomly-initialized models often probe nearly as well as trained ones — is established in the probing literature (Hewitt & Liang 2019; Voita & Titov 2020; Pálsson & Björnsson 2024; §2); our contribution is to instantiate and confirm it for cross-substrate self-play Go, not to discover it. (A genuinely non-local label such as Benson unconditional-life would test learned abstraction more sharply; even our surrounded-point concept partially leaked into the inputs via structure/legality features — input AUC 0.93. A useful additional control would be a deterministic feature expansion [raw features + neighbour mean/max]: since the *untrained* GNN already reaches near-perfect AUC, this would isolate whether the gain comes specifically from local aggregation or from any high-dimensional random nonlinear embedding. `[TODO]`)

5.2 Behavioral transfer (where mixture training matters)

We test whether mixture self-play buys cross-substrate *play* — not just decodable features. For each of **three training seeds** we train, from a fresh initialization under a **matched self-play game and search-simulation budget** (12 generations, 96 self-play simulations, 100 self-play games per generation, 128 search simulations at evaluation), three agents: a **mixture** (the five training substrates), a **square specialist** (square only), and, as an in-domain reference, a **diamond-cubic oracle** (trained directly on the 3-D board). We match the *total* per-generation game budget, not per-substrate exposure: the mixture plays 20 games on each of its five boards (100 total, 20 of them

square), whereas the square specialist plays all 100 on square — so the specialist sees ~5× the square games. We avoid the term "matched compute" because the substrates differ in node count (e.g. 86-node Penrose vs 54-node hexagonal), so per-game forward-pass cost is not equalized; we did not measure FLOPs or node evaluations. Agents then duel head-to-head (60 opening-randomized, colour-alternated games per pairing per seed). Because three seeds cannot estimate training-run variance precisely, **we report each seed's game-level win rate with a 95% Wilson interval, plus the mean and range across seeds, and deliberately do not report a normal-theory "95% CI over seeds"** (a Student-t interval on three points would be ±40–90 points — uninformative).

Table 2 — mixture vs square-specialist, from scratch (per-seed win %, with 95% Wilson game-level interval; three independent training seeds, n = 60 games/seed).

classification	board	seed 0	seed 1	seed 2	mean [range]
exact training condition	square 8×8	18% [11,30]	5% [2,14]	75% [63,84]	33% [5,75]
same-family size transfer	square 7×7	35% [24,48]	25% [16,37]	68% [56,79]	43% [25,68]
same-family patch/scale transfer	Penrose r8	63% [51,74]	100% [94,100]	93% [84,97]	**86% [63,100]**
unseen planar substrate family	trihexagonal	17% [9,28]	25% [16,37]	83% [72,91]	42% [17,83]
unseen non-planar 3-D family	diamond-cubic	55% [42,67]	77% [65,86]	90% [80,95]	**74% [55,90]**

We are careful to classify each board by *what kind* of transfer it tests. Penrose r8 is **not** an unseen family: the mixture trained on Penrose r5, so r8 is a same-family patch/scale generalization. The genuinely unseen families are **trihexagonal** (planar) and **diamond-cubic** (non-planar 3-D).

In the three observed training runs, the mixture outperformed the square specialist on **Penrose r8 and diamond-cubic in every seed** (all three per-seed rates > ½; under a sign test, 3/3 gives a one-sided p = 0.125 — consistent direction, not a significance claim). This supports **strong same-family generalization** (Penrose r5→r8) and **consistent observed transfer to the unseen diamond-cubic family**. On the genuinely unseen *planar* family, **trihexagonal is inconclusive** (two of three seeds below ½). The **square-family** comparisons are likewise **inconclusive at this sample size** — we do not claim equivalence; the point estimate on square 8×8 (33%) in fact suggests a potentially substantial square tax whose magnitude is highly uncertain.

Diamond-cubic, against the non-learned floor. The from-scratch mixture's win rate against the heuristic floor was **82% on average — two runs decisive (98%, 100%), one approximately even (48%)**. The diamond-cubic oracle was similar (mean 83%; one run floor-level at 52%). Separately — and labeled as an *exploratory artifact demonstration*, not part of the three-seed study — the stronger released champion (a selected checkpoint from a different training campaign) clears the floor ~100% on all nine substrates.

Mixture vs the in-domain oracle. The direct head-to-head is the cleaner 3-D result: the mixture **defeated the square specialist on diamond-cubic in each of the three seed pairings** (55 / 77 / 90%). Against the in-domain oracle, both zero-shot agents generally performed poorly:

Table 3 — diamond-cubic, per-seed win % vs the in-domain oracle (n = 60 games/seed).

zero-shot agent vs oracle	seed 0	seed 1	seed 2
square specialist	5%	23%	17%
mixture	5%	15%	87%

The mixture was competitive in only one of three runs (seed 2), and in seed 1 the square specialist actually did slightly better against the oracle than the mixture did. We therefore do **not** claim the mixture is a "far better" 3-D player than the specialist on the oracle comparison — the direct head-to-head is what carries that claim. We also note the across-run variance is large in *absolute* strength even where direction is consistent: seed 2 is a strong run for the mixture across the board (75% on square 8×8, 68% on 7×7, 93% on Penrose r8, 83% on trihexagonal, 90% on diamond-cubic, 87% vs the oracle), while seeds 0–1 are markedly weaker — which is exactly why three seeds cannot pin down the square-tax magnitude and why we report ranges rather than a population interval.

Honest reading. Where the topology genuinely differs (Penrose r8, diamond-cubic), the direction was **consistent across the three evaluated seeds**; we treat that consistency as preliminary descriptive evidence, not a statistically robust effect. Training-seed variance at this 12-generation budget is large (square 8×8 swings 5%→75% across seeds), so the square-family and oracle-margin comparisons are inconclusive. More seeds and/or longer training — most needed on the square family and trihexagonal — would settle them; paired (same-opening, colour-swapped) duels would further cut the per-pairing variance we did not remove here.

5.3 Where transfer breaks (preliminary observations)

A continued-training campaign shows instability/forgetting: a run selected on training boards regressed held-out Penrose r8 to 31% and training-set hexagonal/triangular to 29% — improving some boards eroded others. Two enlargement attempts (champion-distilled 2.6× net; Net2Net expansion) did **not** improve transfer (20–47% vs the teacher/champion). We state this carefully: these are *preliminary* observations of optimization/forgetting/distillation difficulty; they do **not** isolate model capacity (matched-compute, multi-seed, from-scratch scaling curves would be required for a capacity claim).

6. Discussion

The representation that makes tactical variables portable across substrates is largely the architecture and engineered inputs, not a learned abstraction — a still-useful finding: a coordinate-free, topology-aware encoder *plus* rule-derived features already exposes local tactics consistently across square, aperiodic, and 3-D graphs. What mixture training buys is **behavioral**: across the three observed runs the mixture beat a square specialist on the unseen non-planar diamond-cubic family and on a larger Penrose patch — in every seed — whereas transfer to the unseen planar trihexagonal family was inconclusive. The cleanest claim is unseen-family transfer to diamond-cubic; the open problem is acquiring (rather than slowly forgetting) that breadth under continued training, and establishing the square-specialization tradeoff at a larger seed count.

7. Limitations

- Absolute strength is low (~8–10 kyu; vital-point reading 0/18); the agent is an instrument.
- The representational result is decodability, not learned-concept emergence (§5.1); even the surrounded-point ("eye-like") concept leaked into inputs; a Benson-life concept is the definitive

follow-up. Probe AUCs use a single dataset seed; the random baseline spans five inits (giving across-init uncertainty), but the trained nets are single checkpoints.

- Behavioral results are multi-seed but only **three** seeds, at a modest 12-generation budget, under a matched *game/search-simulation* budget (not matched FLOPs or square exposure — the specialist sees ∼5× the square games). Training-seed variance is large, so square-family and trihexagonal and oracle-margin comparisons are inconclusive; we report per-seed Wilson intervals and ranges rather than a normal-theory CI over three seeds, and duels are colour-alternated rather than same-opening paired. The clean positive is unseen-family transfer to diamond-cubic, consistent across three runs.
- The heuristic floor is weak; per-substrate Elo-vs-floor saturates (every agent wins ∼100%), so it does not calibrate absolute strength — a KataGo-anchored ladder is the fix. `[TODO]`
- The diamond patch is small and boundary-heavy (§4).

8. Reproducibility

`uv run python scripts/paper_tables.py --all`, `scripts/probe_decider.py`, and `scripts/probe_table1_std.py` (Table 1 mean ± std, with a seeded random-init baseline) regenerate every table from committed checkpoints; output is deterministic given the seed; `manifest.json` records checkpoint SHA-256s, games/sims, and duel counts. Engine, weights, and scripts are open source; runtimes cross-verified to $\sim 10^{-6}$.

9. Conclusion

A coordinate-free, topology-aware Go agent represents tactical variables consistently across graph substrates including a non-planar 3-D lattice — but a feature-leakage control shows this is mostly architectural, not a learned abstraction. Behaviorally, mixture self-play produced consistent transfer across the three observed runs to an unseen diamond-cubic graph family and strong same-family generalization to a larger Penrose patch; transfer to the unseen planar trihexagonal family and the magnitude of any square-specialization tradeoff remain unresolved at the current experimental scale. The board is data; what *generalizes* across boards is, on the current evidence, the agent's play more than a newly-learned internal concept.